

Scenario 1: Pretrained Model Predicts Only One Class in Object Detection

Context: You fine-tuned a pretrained YOLOv5 model on a custom dataset (detecting helmets). After training, it always predicts only one class.

Problem: Despite having multiple classes in the dataset, the model outputs only “helmet” for every object
Logical Solution:

1. Check the **label format and dataset annotations**; other classes might be mislabeled or missing.
2. Ensure the number of classes is correctly passed to the model.
3. Dataset may be **imbalanced**, so the model is biased toward one class.

Scenario 2: Transfer Learning Model Trains Too Slowly on Small Dataset

Context: You are using a pretrained **ResNet50** model for face mask classification (2 classes). You fine-tune the entire model on a small dataset (500 images).

Problem: Training takes too long and doesn't converge well.

Logical Solution:

1. Instead of fine-tuning the entire model, **freeze early layers** and train only the final classification layer.
2. Use pretrained weights as feature extractor instead of full retraining.

Scenario 3: Transfer Learning Works on Training Data but Fails on New Environment

Context: You used a pretrained YOLOv8 model trained on outdoor images to detect tigers. But when used in night-mode CCTV footage, detection fails.

Problem: Domain shift between training data (bright daylight) and real-world application (night).

Logical Solution:

1. Perform **domain-specific fine-tuning** by augmenting or collecting night-time images.
2. Apply **brightness & contrast augmentation** or use real images under different lighting.

Scenario 4: Object Detection Model Shows Bounding Boxes but Misses Object

Context: You trained a YOLOv5 model using transfer learning on a small vehicle dataset. Boxes are drawn, but not aligned with objects.

Problem:Bounding box coordinates are incorrect due to **wrong label format or image resizing** mismatch.

Logical Solution:

1. Ensure label files follow YOLO format: `[class_id, x_center, y_center, width, height]` in **normalized form**.
2. Match **training image size and validation/test image size**.

Scenario 5: Model Gives High mAP on Training But Low Real-Time Accuracy

Context: You used pretrained **YOLOv8** for multi-class object detection (helmet, vest, mask). mAP on test set is 90%, but real-time inference on webcam is inaccurate.

Problem:Difference in resolution, camera angle, or image quality between webcam and training data.

Poor generalization due to **dataset bias**.

Logical Solution:

1. Calibrate webcam feed: resize to same resolution as training.
2. Collect some real-time webcam samples and **fine-tune the model**.
3. Apply **real-time test-time augmentation (TTA)** techniques.

Scenario 6: VGG16 Model Very Slow During Training

Project: Fashion product classification using **VGG16**

Problem:Model takes too long to train even with a small dataset and batch size.

Logical Solution:

1. VGG is computationally heavy. Try **feature extraction** mode by freezing all convolutional layers.
2. Replace the fully connected (FC) layers with a lighter classifier.

Scenario 7: InceptionV3 Fails to Train Due to Input Size Mismatch

Project: Dog breed classification using **InceptionV3**

Problem:Runtime error due to input size mismatch.

Logical Solution:

1. InceptionV3 expects **input size of 299x299** (not 224x224 like ResNet/VGG).
2. Resize all training and validation images accordingly.